

أولاً: مفهوم لغة التجميع وعملية التشغيل (Assembly Process)

1. التعريف:

• هي لغة برمجة منخفضة المستوى (Low-level) تعتمد على المعمارية (Machine-dependent).

• يتم تحويلها إلى لغة آلة (Machine Code) باستخدام برنامج يسمى **Assembler**.

2. خطوات تحويل وتنفيذ البرنامج (مهم للترتيب في الامتحان):

Editor: 1. كتابة الكود وحفظه بصيغة .asm.

Assembler: 2. يترجم الكود وينتج ملف **Object File (.OBJ)** يقوم بحساب الـ Offsets لكل تعليمة.

Linker: 3. يربط ملفات الـ OBJ المختلفة ويحولها إلى ملف تنفيذي (.EXE) جاهز للتشغيل.

Loader: 4. يحمل البرنامج في الذاكرة للتنفيذ ويقوم بإنشاء **PSP (Program Segment Prefix)**.

ثانياً: أنواع البرامج (.EXE vs .COM)

هذه مقارنة جوهرية في المحاضرة:

	.COM Programs	.EXE Programs	الخاصية
	محدود بـ 64KB فقط.	غير محدود (Unlimited size)	الحجم
	يدمج كل الـ (Code, Data, Stack) في Segment واحد فقط .	يحتوي على Stack, Data, Code على segments منفصلة.	Segments
	أصغر وأسرع تحميلاً.	أبطأ قليلاً بسبب الـ Header.	السرعة
	الـ Loader يوجه كل المسجلات لبداية الـ PSP (ORG 100H) المبرمج يحتاج.	يحتاج تهيئة للـ Segments.	التهيئة (Initialization)

ثالثاً: تقسيم الذاكرة والمسجلات (Segments & Registers)

1. الـ Segments الأساسية:

Code Segment (CS): يحتوي على تعليمات البرنامج (Instructions).

Data Segment (DS): يحتوي على البيانات والمتغيرات والثوابت.

Stack Segment (SS): يحتوي على العناوين والبيانات المؤقتة للـ Subroutines.

2. المسجلات العامة: (General Purpose Registers)

AX (Accumulator): للعمليات الحسابية والـ I/O.

BX (Base): للعناوين (Addressing).

CX (Counter): يُستخدم كعداد في عمليات التكرار (Looping) سؤال امتحان متكرر.

DX (Data): لعمليات الضرب والقسمة و I/O.

3. مسجلات الأعلام (Flag Register):

ZF (Zero Flag): 1 لو الناتج صفر.

SF (Sign Flag): 0 للموجب، 1 للسالب.

TF (Trap Flag): يستخدم للـ (Debugging (Step-by-step execution) هام جداً).

OF (Overflow): طيح في العمليات الحسابية.

رابعاً: بناء جملة الـ Assembly (Syntax)

1. المكونات:

Identifier: الاسم (Label) ومسموح فيه حروف وأرقام (بشرط عدم البدء برقم) ورموز خاصة مثل ? و _ و @. أقصى طول 247 حرف.

Comments: تبدأ بـ (;) Fasla Manqota.

2. تعريف البيانات (Data Definition):

• DB: Define Byte (8 bits).

• DW: Define Word (16 bits).

• DD: Define Double Word (32 bits).

DUP: لتكرار القيمة (مثال DW 10 DUP(?)) تحجز 10 أماكن فارغة.

3. الـ Operands المعاملات:

• التعليمات قد لا تأخذ معامل (RET)، أو تأخذ واحد (INC)، أو اثنين (MOV)، أو ثلاثة.

• مثال هام للامتحان: التعليمة SHRD تأخذ 3 Operands مثل SHRD ECX, EBX, CL.

خامساً: الفرق بين Directives و Instructions

هذا الجزء هو الأكثر تكراراً في أسئلة الـ MCQ

1. Instructions (التعليمات):

• تتحول إلى Machine Code.

• أوامر للمعالج ليقوم بعملية (نقل، جمع، طرح).

• أمثلة: MOV, ADD, SUB, INC, DEC.

2. Directives (التوجيهات): Pseudo-ops)

• لا تولد. Machine Code

• هي أوامر للمجمع (Assembler) نفسه لتنظيم البرنامج.

• أمثلة:

SEGMENT ◦ و ENDS: لبدء ونهاية الـ Segment.

PROC ◦ و ENDP: لتعريف دالة (Procedure).

ASSUME: ◦ لربط الـ Logical segments بالـ Physical registers.

END: ◦ لنهاية البرنامج بالكامل.

◦ PAGE, TITLE.

أهم نقاط الامتحانات (MCQ Summary for Lec 7&8)

1. Looping Register: CX. هو الـ

2. Tracking Instructions: المسجل المسؤول عن تتبع التعليمات هو PC (Program Counter) أو IP.

3. Directives: "Generate NO machine code" وهي توجيهات للـ Assembler.

4. SHRD Instruction: هي تعليمة تأخذ 3 Operands.

5. Comments: تبدأ بـ ; وتشرح وظيفة البرنامج.

6. Reserved Words: كلمات محجوزة لا يمكن استخدامها كأسماء (مثل MOV, ADD, SIZE, FAR).

7. Data Definition: DW تستخدم لتعريف Word.

8. COM Programs: حجمها محدود بـ 64 KB وتدمج الـ Segments